Trabajo Práctico Nº 2

Introducción al trabajo con hilos: Thread / Runnable

1. En Java, ejecutar el siguiente código **repetidas** veces:

Cliente.java

```java
public class Cliente extends Thread {
    private Recurso miRecurso;

    public Cliente(Recurso miRecurso) {
        this.miRecurso = miRecurso;
    }

    public void run() {
        System.out.println("soy" + Thread.currentThread().getName());
        this.miRecurso.uso();
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            System.out.println("Error");
        }
    }
}
```

Recurso.java

```java
public class Recurso {
    public Recurso() {}

    public void uso() {
        Thread t = Thread.currentThread();
        System.out.println("en Recurso:  Soy" + t.getName());
    }
}
```

TesteoRecurso.java

```java
public class TesteoRecurso {
    public static void main(String[] args) {
        Recurso unRecurso = new Recurso();

        Cliente juan = new Cliente(unRecurso);
        juan.setName("Juan Lopez");
        Cliente ines = new Cliente(unRecurso);
        ines.setName("Ines Garcia");

        juan.start();
        ines.start();
    }
}
```

a) Analice el funcionamiento del siguiente código. ¿Cuántos hilos de control participan en la ejecución?

b) ¿Cúal es la funcionalidad del método "uso" de `Recurso`?

c) Indique todas las salidas posibles.

d) ¿Qué sucede si agrega la línea "`unRecurso.uso()`" al final del `main`?

e) ¿Qué sucede si cambia el orden de las instrucciones "`start()`"? ¿Tiene algún impacto en el resultado de la ejecución?

2. Ejecutar el siguiente código en **repetidas veces**:

Hilo.java

```java
public class Hilo extends Thread {
    public void run() {
        ir();
    }

    public void ir() {
        hacerMas();
    }

    public void hacerMas() {
        System.out.println("En la pila");
    }
}
```

TesteoHilos.java

```java
public class TesteoHilos {
    public static void main(String[] args) {
        Thread miHilo = new Hilo();
        miHilo.start();

        System.out.println("En el main");
    }
}
```

a) ¿Cómo es el comportamiento de las diferentes ejecuciones?

b) ¿Se podría forzar las ejecuciones para que se comporte de una manera determinada? Realice las modificaciones necesarias.

3. Ejecute el siguiente programa:

ThreadEjemplo.java

```java
public class ThreadEjemplo extends Thread {
    public ThreadEjemplo(String str) {
```

```
        super(str);
    }

    public void run() {
        for (int i = 0; i < 10 ; i++)
            System.out.println(i + " " + getName());

        System.out.println("Termina thread " + getName());
    }
}
```

MainThreadEjemplo.java

```
public class MainThreadEjemplo {
    public static void main(String[] args) {
        new ThreadEjemplo("Maria Jose").start();
        new ThreadEjemplo("Jose Maria").start();

        System.out.println("Termina thread main");
    }
}
```

a) Si se ejecuta varias veces, ¿qué sucede?

4. La interfaz **Runnable** proporciona un método alternativo a la utilización de la clase **Thread**, implemente el ejercicio anterior utilizando la interfaz **Runnable**.

a) ¿Qué sucede? ¿Es necesario el constructor?

5. Analizar el siguiente código:

HiloContador.java

```
public class HiloContador implements Runnable {
    String nombreHilo;

    public HiloContador(String nombre) {
        nombreHilo = nombre;
    }

    // Punto de entrada del hilo
    // Los hilos comienzan a ejecutarse aquí
    public void run() {
        System.out.println("Comenzando " + nombreHilo);
        try {
            for (int contar = 0; contar < 10; contar++) {
                Thread.sleep(400);
                System.out.println("En " + nombreHilo
                        + ", el recuento " + contar);
            }
```

```
        } catch (InterruptedException exc) {
            System.out.println(nombreHilo + " Interrumpido");
        }
        System.out.println("Terminando " + nombreHilo);
    }
}
```

UsoHilos.java

```
public class UsoHilos {
    public static void main(String[] args) {
        System.out.println("Hilo principal iniciando.");

        // Primero, construye un objeto unHilo.
        HiloContador hc = new HiloContador("#1");

        // Luego, construye un hilo de ese objeto.
        Thread nuevoHilo = new Thread(hc);

        // Finalmente, comienza la ejecución del hilo.
        nuevoHilo.start();

        for (int i = 0; i < 50; i++) {
            System.out.print(" .");
        }

        try {
            Thread.sleep(100);
        } catch (InterruptedException exc) {
            System.out.println("Hilo principal interrumpido");
        }
        System.out.println("Hilo principal finalizado");
    }
}
```

a) ¿Qué pasaría si quitamos todos los `sleep()` del programa? ¿Y si solo quitamos el de `main`, o el de `HiloContador`? Para cada caso, ¿cuál sería la salida del programa?

b) El `main()` ¿siempre termina al final, o puede suceder que termine antes que el `run()`? ¿Por qué puede suceder esto?

c) Realizar los cambios necesarios para que la clase `HiloContador` herede de la clase `Thread` (recuerde eliminar la interfaz `Runnable`).

d) Realice el ejercicio para que en vez de ejecutar un solo hilo, ejecute 3. Realice el procedimiento varias veces. ¿Qué ocurre? ¿Se ejecutan en orden?

6. Simulación de Carrera Multithread

Imagine que está organizando una carrera de atletismo con varios corredores. Usted quiere simular esta carrera utilizando múltiples hilos en Java. Cada corredor será representado por un hilo independiente que avanzará en la pista de carreras. El primer paso es crear una clase

llamada **Corredor** que serán los hilos. Dentro de esta clase, defina los atributos necesarios como el nombre del corredor y la distancia recorrida.

Cada corredor sabe la distancia que recorrió, por lo tanto, en cada paso que realiza deberá imprimir su nombre y la distancia que pudo avanzar (aleatorio entre 1 y 10). Luego de realizado el paso debe descansar. Una vez alcanzado los 100 pasos, el corredor termina.

En la clase principal, debe crear un arreglo de 100 objetos `Corredor` (instancias de la clase `Corredor`). Para cada corredor en el arreglo, debe crear un objeto `Thread` y pasar la instancia del corredor como argumento al constructor del `Thread`.

Debe iniciar todos los hilos creados usando el método `start()`. Utilice `Thread.sleep()` dentro del método `run()` de cada corredor para simular el tiempo entre pasos.

Al finalizar la competencia de todos los atletas, el programa deberá determinar e informar por consola qué corredor logró la mayor distancia recorrida y cuál fue dicha distancia.

7. Supongamos que debemos simular el proceso de cobro de un supermercado; es decir, unos clientes van con un carro lleno de productos y un cajero les cobra los productos, pasando uno a uno por el escáner de la caja registradora. En este caso el cajero debe procesar la compra cliente a cliente, es decir que primero le cobra al cliente 1, luego al cliente 2 y así sucesivamente.

Para ello se debe definir una clase `Cajero` y una clase `CompraCliente` el cual tendrá un *array de enteros* que representarán los productos que ha comprado y el tiempo que el cajero tardará en pasar el producto por el escáner. Por ejemplo, si tenemos un array con `[1, 3, 5]` significa que el cliente ha comprado 3 productos y que el cajero procesará el producto 1 en "1 segundo", el producto 2 en "3 segundos" y el producto 3 en "5 segundos", con lo cual el tiempo total empleado por el cajero será de 9 segundos.

a) El siguiente código simula la operación de cobro con dos clientes con un solo proceso (que es lo que se suele hacer normalmente). Teniendo en cuenta que se procesa primero la compra del Cliente 1 y después la del Cliente 2, con lo cual se tardará el tiempo del Cliente 1 + Cliente 2.

Completar y ubicar en la clase que corresponda la implementación del método: `esperarXsegundos`.

Cajero.java

```java
public class Cajero {
    private String nombre;

    // TODO: Agregar Constructor, y métodos de acceso

    public void procesarCompra(CompraCliente compraCliente, long timeStamp) {

        System.out.println("El cajero " + this.nombre
                + " COMIENZA A PROCESAR LA COMPRA DEL CLIENTE "
                + compraCliente.getNombre() + " EN EL TIEMPO: "
                + (System.currentTimeMillis() - timeStamp) / 1000 + " seg");

```

```
        for (int i = 0; i < compraCliente.getProductos().length; i++) {
            this.esperarXsegundos(compraCliente.getProductos()[i]);
            System.out.println("Procesado el producto " + (i + 1)
                + " -> Tiempo: "
                + (System.currentTimeMillis() - timeStamp) / 1000 + " seg");
        }

        System.out.println("El cajero " + this.nombre
                + " HA TERMINADO DE PROCESAR "
                + compraCliente.getNombre() + " EN EL TIEMPO: "
                + (System.currentTimeMillis() - timeStamp) / 1000 + " seg");
    }
}
```

CompraCliente.java

```
public class CompraCliente {
    private String nombre;
    private int[] productos;

    // TODO: Agregar constructor y métodos de acceso
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {

        CompraCliente compra1 = new CompraCliente("CompraCliente1",
                new int[] { 2, 2, 1, 5, 2, 3 });
        CompraCliente compra2 = new CompraCliente("CompraCliente2",
                new int[] { 1, 3, 5, 1, 1 });
        Cajero cajero1 = new Cajero("Cajero1");

        // Tiempo inicial de referencia
        long initialTime = System.currentTimeMillis();
        cajero1.procesarCompra(compra1, initialTime);
        cajero1.procesarCompra(compra2, initialTime);
    }
}
```

Salida:

```
El cajero Cajero1 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE CompraCliente1 EN EL TIEMPO: 0 seg
Procesado el producto 1 -> Tiempo: 2 seg
Procesado el producto 2 -> Tiempo: 4 seg
Procesado el producto 3 -> Tiempo: 5 seg
Procesado el producto 4 -> Tiempo: 10 seg
Procesado el producto 5 -> Tiempo: 12 seg
Procesado el producto 6 -> Tiempo: 15 seg
El cajero Cajero1 HA TERMINADO DE PROCESAR CompraCliente1 EN EL TIEMPO: 15 seg
El cajero Cajero1 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE CompraCliente2 EN EL TIEMPO: 15
    seg
Procesado el producto 1 -> Tiempo: 16 seg
Procesado el producto 2 -> Tiempo: 19 seg
Procesado el producto 3 -> Tiempo: 24 seg
```

```
Procesado el producto 4 -> Tiempo: 25 seg
Procesado el producto 5 -> Tiempo: 26 seg
El cajero Cajero1 HA TERMINADO DE PROCESAR CompraCliente2 EN EL TIEMPO: 26 seg
GENERACIÓN CORRECTA (total time: 26 seconds)
```

b) ¿Y si en vez de procesar primero un cliente y después otro, procesamos los dos a la vez? ¿Cuánto tardaría el programa en ejecutarse? Si en vez de haber solo un cajero (es decir un solo hilo), hubiese dos cajeros (es decir dos hilos o threads) podríamos procesar los dos clientes a la vez y tardar menos tiempo en ejecutar el programa. Complete el siguiente código a fin de representar el escenario descrito.

CajeroThread.java

```java
public class CajeroThread extends Thread {
    private String nombre;
    private CompraCliente compraCliente;
    private long initialTime;

    // TODO: Agregar constructor, y métodos de acceso

    public void run() {

        System.out.println("El cajero " + this.nombre
                + " COMIENZA A PROCESAR LA COMPRA DEL CLIENTE "
                + this.compraCliente.getNombre() + " EN EL TIEMPO: "
                + (System.currentTimeMillis() - this.initialTime) / 1000
                + "seg");

        for (int i = 0; i < this.compraCliente.getProductos().length; i++) {
            this.esperarXsegundos(compraCliente.getProductos()[i]);
            System.out.println("Procesado el producto " + (i + 1)
                + " del cliente " + this.compraCliente.getNombre()
                + " -> Tiempo: "
                + (System.currentTimeMillis() - this.initialTime) / 1000
                + "seg");
        }

        System.out.println("El cajero " + this.nombre
                + " HA TERMINADO DE PROCESAR "
                + this.compraCliente.getNombre() + " EN EL TIEMPO: "
                + (System.currentTimeMillis() - this.initialTime) / 1000
                + "seg");
    }
}
```

MainThread.java

```java
public class MainThread {
    public static void main(String[] args) {

        CompraCliente compra1 = new CompraCliente("CompraCliente1",
                new int[] { 2, 2, 1, 5, 2, 3 });
        CompraCliente compra2 = new CompraCliente("CompraCliente2",
                new int[] { 1, 3, 5, 1, 1 });

```

```
        // TODO: Completar main
    }
}
```

Posible salida:

```
El cajero Cajero1 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE CompraCliente1 EN EL TIEMPO: 0 seg
El cajero Cajero2 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE CompraCliente2 EN EL TIEMPO: 0 seg
Procesado el producto 1 del cliente CompraCliente2 -> Tiempo: 1 seg
Procesado el producto 1 del cliente CompraCliente1 -> Tiempo: 2 seg
Procesado el producto 2 del cliente CompraCliente1 -> Tiempo: 4 seg
Procesado el producto 2 del cliente CompraCliente2 -> Tiempo: 4 seg
Procesado el producto 3 del cliente CompraCliente1 -> Tiempo: 5 seg
Procesado el producto 3 del cliente CompraCliente2 -> Tiempo: 9 seg
Procesado el producto 4 del cliente CompraCliente1 -> Tiempo: 10 seg
Procesado el producto 4 del cliente CompraCliente2 -> Tiempo: 10 seg
Procesado el producto 5 del cliente CompraCliente2 -> Tiempo: 11 seg
El cajero Cajero2 HA TERMINADO DE PROCESAR CompraCliente2 EN EL TIEMPO: 11 seg
Procesado el producto 5 del cliente Cliente 1 -> Tiempo: 12 seg
Procesado el producto 6 del cliente Cliente 1 -> Tiempo: 15 seg
El cajero Cajero1 HA TERMINADO DE PROCESAR CompraCliente1 EN EL TIEMPO: 15 seg
GENERACIÓN CORRECTA (total time: 15 seconds)
```

8. Realice el ejercicio 7b utilizando la *interfaz Runnable*.